AI Now Writes Lean Proofs Faster Than Lean Can Check Them

Yusheng Zheng
UC Santa Cruz
yzhen165@ucsc.edu

Abstract

As AI gets better at writing mathematical proofs, formal verification is expected to keep checking them cheaply. We test this on OpenAI's AI-generated Lean library and find that AI now writes Lean proofs faster than Lean checks them: Lean compiles the median file at about 400 tokens per CPU-second, so a model emitting 1,000 tokens per second finishes first on 95% of 162,170 files. The cause is the way AI writes proofs, as short instructions that leave search and computation to Lean; the library uses proofs by computation 10,668 times more densely than Mathlib. Storing the finished proof helps ordinary proofs, whose re-check costs 0.5–6.5% of the first check, and helps computational proofs far less, at 80–82%. Checking the library took at least 169 P-core and 302 E-core CPU-hours. Checking AI-written proofs therefore needs the capacity planning that model inference already receives. We release ProofServe, which re-checks Lean projects in parallel with Lean's own kernel and reports what each check covered.

1 Introduction

AI is getting better at writing mathematical proofs, which raises a question: can we keep up with checking what it writes? The usual answer is formal verification. The AI writes its proof as Lean code, Lean checks every step automatically, a proof that passes is correct, and machine checking is fast and cheap [4]. The same expectation lets Lean act as the reward signal when AI provers are trained, where every candidate proof a model writes is checked before the model learns from it [5, 20].

We tested that answer on OpenAI's public library of AI-generated mathematics [19] and found that AI now writes Lean proofs faster than Lean can check them. A model can write a 450-line file in under a minute; Lean, on one processor core, took 2.6 hours to check it. Across the library's 162,170 files, a model writing 1,000 tokens per second, a speed fast inference services already reach for a single stream, finishes first on 95% of them (Figure 1). Formal verification is supposed to be the cheap step that lets us trust AI-written mathematics. For AI-written proofs, it is becoming the expensive one.

If models keep getting faster, the gap will widen. Model output speeds keep rising with better hardware and batched inference, Lean checks each file on essentially one processor core, and AI-written proofs hand more and more of their work to Lean. A model's output is often a short instruction: a tactic that makes Lean search, or a call to `decide` that makes Lean compute. Storing the finished proof and re-checking it later removes the search; the computation remains, because the computation is the proof. We measure this shift and its cost, and make four contributions:

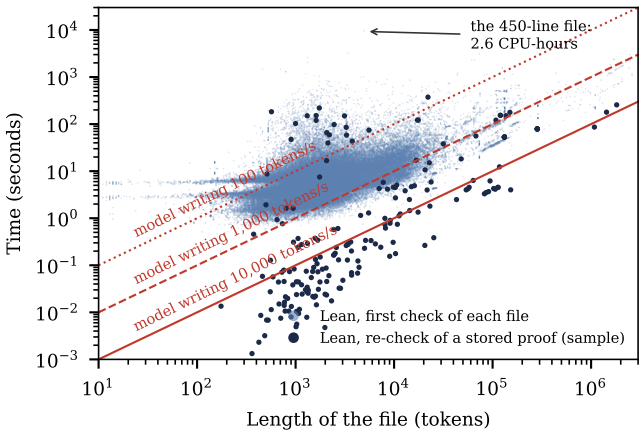


Figure 1: Writing versus checking, per module. Lines: time for a model to write out a file of the given length at 100, 1,000 and 10,000 tokens per second. Light points: Lean's CPU time to compile each of 162,170 modules. Dark points: CPU time to re-check the stored proofs of 200 sampled modules, half drawn uniformly and half from the costliest 1% and from modules containing `decide +kernel`. Points above a line are checked more slowly than a model at that speed writes them.

- **Writing versus checking.** We compare the time a model needs to write out each module with the time Lean needs to compile and to re-check it (§4, Figure 1).
- **Where the work goes.** AI-written proofs leave search and computation to Lean; re-checking costs 0.5–6.5% of compilation for ordinary proofs and 80–82% for computational ones (§5).
- **The bill for one library.** A per-module census of 162,170 compiled modules, its concentration, its avoidable overheads, and the files a default build leaves out (§6).
- **ProofServe**, an open-source tool that re-checks compiled Lean projects in parallel with the original kernel and reports coverage and trust assumptions (§7).

2 Checking a Lean proof

Lean is a proof assistant: proofs are written as source files, much like programs, and Lean confirms that every step follows from the ones before it. Mathlib is the largest human-written library of such proofs. Proof authors usually leave most steps for Lean to find. They write *tactics*, short instructions such as `simp`, `omega` or `linarith` that tell Lean how to find the steps. Lean's elaborator runs the tactics, which may search and backtrack, and produces a complete *proof term* that records only the steps that worked. A small trusted *kernel* then type-checks that term, and trust in the system rests on the kernel.

Checking therefore happens at two moments. The first check of a source file runs every tactic and then the kernel; this is *compiling* the file, and it is the only check available for source text that a model writes. Compilation stores the checked terms in `.olean` files, and a later *re-check* replays those terms through the kernel and skips every tactic, as Lean's `leanchecker` and the `lake check` command of Lean 4.35 do [13, 12].

Re-checking removes search and keeps computation. For a proposition p that can be decided mechanically, `decide` produces a proof whose validity requires the kernel to reduce `decide` p to `true` [10], which means running the decision procedure on the actual finite object. `decide +kernel` leaves that run entirely to the kernel. Either way, the run is part of the kernel check and is repeated by every re-check.

3 Methodology

Corpus and machine. We study OpenAI's repository at commit `4eaee875`: 163,096 Lean modules plus one build configuration, on Lean 4.34.1 and Mathlib `d13f23b7` [19, 6]. The library has about nineteen times as many files as that Mathlib snapshot. All measurements run on one Intel Core i9-14900K with 8 performance (P) cores, 16 efficiency (E) cores, SMT disabled, 64 GB of memory and Linux 7.0. P and E cores do different amounts of work per second, so we report their CPU time separately and keep the two units apart. Other users' workloads kept running on the machine.

Writing time. We count each module's tokens with the `o200k_base` tokenizer (676.6 million tokens in all, 2.64 bytes per token) and divide by a model's output rate. Three reference rates match what large models reach in 2026. In May 2026 the fastest GPU-based provider served Kimi K2.6, a trillion-parameter model, at about 146 tokens per second per stream, and Cerebras served it at 981, as measured by Artificial Analysis [3]; we use 100 and 1,000 tokens per second for these two settings. The third rate, 10,000 tokens per second, is the output of two GPUs in a batched server: each GPU of an NVIDIA GB300 NVL72 rack serving DeepSeek-V4-Pro, a 1.6-trillion-parameter model, produced 6,182 output tokens per second across its users [21]. Writing time counts the final file alone; the reasoning and failed attempts that produced it lie outside it.

Checking time. We compiled every module in dependency order, one process per module with Lean's standard `-j8` setting and up to 22 modules at once on 8 P and 14 E cores, and recorded each module's CPU and wall time, peak memory and instruction counts. Families that repeatedly exhausted memory were deferred with their dependents. For re-checking, we time the kernel replay of one module after its dependencies are loaded, separately from loading them: for four modules matched against their compilation, for 31 modules as complete checker and compiler processes, and for 200 modules sampled uniformly, from the costliest 1%, and from modules containing `decide +kernel`. Timed re-checks run on the P cores with memory limits enforced by cgroups in place of Lean's `-M` option (§6.2 explains why).

Comparing modules across cores. In a calibration of 240 compilations under four conditions (P or E cores, quiet or loaded), the user-mode instruction count of the same module varied by a median of 0.018% (95th percentile 0.14%), while kernel-mode counts varied by a median of 8.6% and repeated CPU time by up to 20%. We therefore compare modules by user-mode instructions, a choice made after total instruction counts missed our preset stability targets (1% median, 3% 95th percentile).

4 Can checking keep up with writing?

A model writing 1,000 tokens per second outpaces Lean's compilation on 95% of the modules (Figure 1). The median module compiled at 396 tokens per CPU-second on a P core and 206 on an E core; over the whole library the rates were 577 and 298, because large certificate files compile more tokens per second than small ones. A model writing 100 tokens per second already outpaces Lean on 8% of the modules (3.5% on P cores, 12.9% on E cores). At 1,000 tokens per second it outpaces Lean on 95% of them (91.8% and 99.1%), and at 10,000 on all of them. Keeping pace with one such GB300 GPU at 6,182 tokens per second would take about 11 P cores of compilation at the library-wide rate, before any re-check.

Re-checking a stored proof is far faster. In a uniform sample of 100 compiled modules, reading, mapping and replaying a module's stored proof took longer than writing it at 1,000 tokens per second for 3 modules and at 10,000 tokens per second for 17. Each re-check first loads the module's imports, a median of 4.0 seconds against a median of 0.05 seconds for the check itself, and with that load counted the two numbers rise to 85 and 100. The other 100 sampled modules, drawn from the costliest 1% and from modules containing `decide +kernel`, fall behind more often: across all 200 modules, 17.5% and 44% of re-checks took longer than writing at the two rates. One sampled module of 569 tokens took 184 seconds to re-check, while writing it at 100 tokens per second takes 5.7 seconds. Across the 200 modules, re-checking ran at about 2,040 tokens per CPU-second.

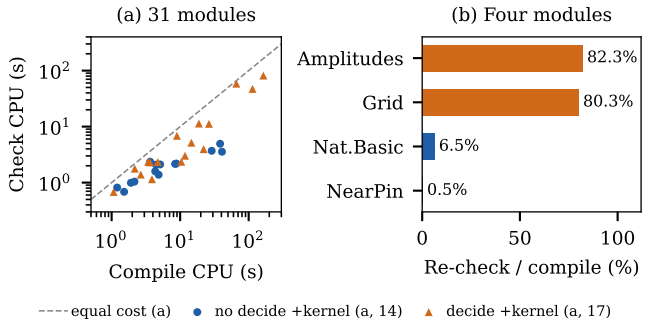


Figure 2: Re-checking relative to compilation. (a) CPU time of complete checker and compiler processes for 31 modules, by whether the source contains `decide +kernel`; the groups overlap. (b) Time to re-check one module after its dependencies are loaded, as a share of its compilation wall time: ordinary proofs (blue) and computational certificates (orange).

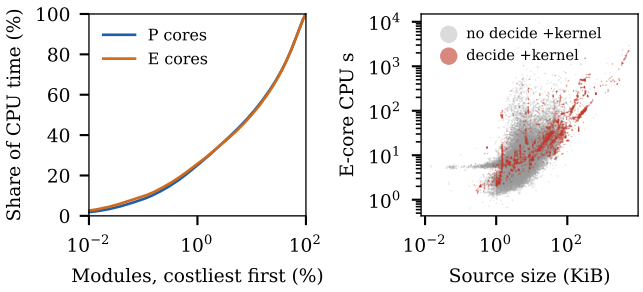


Figure 3: Compilation cost across 162,170 modules. (a) Cumulative share of CPU time against the share of modules, costliest first. (b) Source size against E-core CPU time; modules containing `decide +kernel` are red.

5 Where the work goes

5.1 AI-written proofs compute

After masking comments and strings, the AI library contains 1,354,379 `decide +kernel` calls in 5,848 files. The Mathlib snapshot it builds on contains seven, in four of its 8,529 files. Normalized by source size, the AI library uses this construct 10,668 times more densely. `Grid`, a certificate with 590 lemmas and 1,664 such calls, is typical:

```
lemma p0_0_contains :
    p0_0.Contains (quadraturePoint 0 0) := by
  ...
  have he : PointBox.calc p0_0.t = p0_0 := by
    decide +kernel
```

The step is one line, but checking it makes the kernel redo interval arithmetic on 30-digit rationals. Why the AI library computes so much remains an open question. A likely factor is that many of its results concern specific finite objects, where exhaustive evaluation is a proof that search finds reliably.

Search is the other route. The 452-line file from the introduction, `OAI.Analysis.FreeWords.UltraExpectationFull`, is the costliest module in the library at 9,314 CPU-seconds on an E core, and its source contains zero `decide` calls; locating where its time goes needs a profile we leave to future work.

5.2 Re-checking repeats the computation

Re-checking removes search, so for ordinary proofs it is cheap. Measured after dependencies are loaded (Figure 2(b), wall time), Mathlib's `Nat.Basic` re-checks in 6.5% of its compilation time and the AI library's `NearPin` in 0.5%. For computational certificates the computation is the proof: `Grid` and `Amplitudes` re-check in 80.3% and 82.4% of their compilation time, 255.5 and 110.0 seconds almost entirely inside the kernel's replay call.

Measured as whole processes (Figure 2(a), CPU time), the gap narrows: across 31 modules a checker process used a median 45% of the compiler process's CPU time, 50% for the 17 modules containing `decide +kernel` and 39% for the other 14. Both processes load the same imports first, and for short modules that shared load is most of the cost (§6.2). Plain `decide` and other tactics also leave evaluation in terms; a Ramsey K_{10} certificate free of `decide +kernel` needed 356 CPU-seconds to re-check against 1,725 to compile. Averages over a project hide such modules: re-checking the full dependency closure of the `Falconer` development cost 6.3% of its compilation CPU, because many cheap dependencies dilute a few expensive certificates.

6 The bill for one library

6.1 Compiling 162,170 modules

Compiling the library took at least 169 P-core and 302 E-core CPU-hours, and a quarter of each went to 1% of the modules. Of 163,096 modules, 162,170 (99.43%) compiled, every one successfully. We deferred 926 because a few families, including `PadicHodge`, kept exhausting the memory of their build jobs; we set those families aside with every module that depends on them. The completed part took 31.5 hours of wall time, 169.0 P-core and 302.4 E-core CPU-hours. These are lower bounds: the deferred modules are the most memory-hungry ones, and the completed set follows dependency order, which differs from a random sample.

Figure 3 shows the concentration. The median module needed 4.0 CPU-seconds on a P core and 7.5 on an E core. The 5,848 files containing `decide +kernel` are 3.6% of the modules, account for 18% of the CPU time and make up 41–45% of the costliest 1%. Source size predicts cost only moderately (Spearman rank correlation 0.60–0.62). The median compile process peaked at 6.2–6.4 GiB resident and the largest at 25 GiB, and 65 compile attempts were stopped by memory limits before succeeding on retry or joining the deferred families. An earlier report built 122,458 modules of an older revision in 13.8 hours on 32 cloud vCPUs and estimated about 410 CPU-hours, including roughly 330 extrapolated from 26 timed modules [18]; the census replaces that extrapolation with a measured cost per module.

6.2 Avoidable overhead

Part of the bill is overhead around the checking itself. Every Lean process first loads its imports: for 13 short modules this took a median of 1.36 seconds and 3.1 GiB, 68% of their build time, and the official checker took 1.28 times as long as the build on the same modules. A profile of importing Mathlib attributes the time to marking objects persistent (9.3%), name comparison (5.7%), environment hashing and memory allocation. Lean's `-M` memory limit [11] makes Lean call `getrusage` [16] on every 200th memory check, which the kernel runs throughout reduction; removing it and enforcing the same 15 GiB through a cgroup [15] cut `Grid`'s compile time from 622 to 306 seconds with byte-identical output. Finally, `leanchecker` resolves every `.olean` on its search path before selecting the requested root [9]: in the library's 90-entry build path, checking `Nat.Basic` took 322.9 seconds, against 0.39 seconds with a path holding only its 1,330 imported modules.

6.3 What a successful build covers

A successful default build compiles only the modules reachable from the declared root targets. In this library those roots reach 130,834 of 163,097 source units. The other 32,263 files (19.8%) lie outside every default build, and they hold 451,008 of the 1,354,379 `decide +kernel` calls (33.3%). The earlier report found about a quarter of the files outside the default graph at its revision [18]. A claim that such a library "checks" should state which roots were checked.

7 ProofServe

ProofServe turns these lessons into a re-checking tool. `proofserve check` takes a compiled project, finds its roots and import closure, and reports missing artifacts. Within the closure it builds a graph of declarations from the constants they reference and checks ready declarations in parallel waves, each through Lean's original kernel against the already-checked frontier, marking newly checked objects persistent between waves. Declarations other than ordinary definitions and theorems go through Lean's own replay path first. Checkpointed segments let an interrupted check resume, and a receipt records versions, the module set, accepted and rejected modules, their declared axioms, and CPU and wall time. `proofserve audit` reports which files and `decide +kernel` calls the default roots miss and what a receipt covered.

ProofServe trusts Lean's kernel, its artifact reader, the coordinator and local checkpoints. By default it reuses a kernel-checked theorem body when another module declares a theorem with the same interface, which saves work but can accept a crafted artifact that official checking rejects in its declared import context; `-strict-theorem-bodies` checks each body in its own context.

On three closures (Table 1), eight workers cut wall time by 41–53% (speedups of 1.70, 2.14 and 1.73) while total CPU rises by 2.9, 11.3 and 4.6%. These are closure-level results: a re-check of the whole library ran past our time budget. Strict mode costs 4.23 times the wall time on `Nat.Basic`; on `Grid` it timed out at 1,800 seconds. Turning off persistent marking adds 12–17% CPU, and an interrupted check resumed with 17% extra wall time and the same result. On three external projects, including `samuelburbulla/proofs`, a collection its authors describe as AI-generated, ProofServe and the official checker accepted all 126 roots they share; ProofServe checked the full closures while the official default mode checked only the projects' own modules, so their times measure different amounts of work.

		Workers			
Closure	Clock	1	2	4	8
`Nat.Basic`	CPU	110.3	111.3	112.1	113.5
1,330 modules	Wall	110.8	82.7	69.7	65.3
`Grid`	CPU	751.2	778.6	799.1	835.8
3,793 modules	Wall	754.0	517.3	398.4	352.4
`Barrier`	CPU	732.6	–	–	766.3
4,449 modules	Wall	735.6	–	–	425.9

Table 1: ProofServe check time in seconds by number of workers (median of three fresh runs, default mode).

8 Implications

If models keep writing proofs faster than Lean checks them, verification needs the planning that model inference already receives. Give checking its own capacity next to generation and budget it in tokens per CPU-second: at the library-wide rate, keeping pace with one GB300 GPU serving a frontier model takes about 11 P cores of compilation. Budget computational certificates individually, and count how much of a library relies on computation before sizing a check. Report which roots a build or check covered. Load imports once per worker and reuse them for every file, keep search paths small, and enforce memory limits outside Lean.

9 Limitations

We compare Lean's time with the time to write out the final file. Producing that file took the AI system more tokens, including reasoning and failed attempts; that cost lies outside our data, so our claim concerns the time to write out the final file. Re-checking ordinary stored proofs remains cheap. All results come from one machine, one revision of one library and Lean 4.34.1; the census is a lower bound over a dependency-ordered subset that excludes the most memory-hungry families; `decide +kernel` counts are lexical proxies; and the overhead measurements of §6.2 are few or single observations. A complete re-check of the library with ProofServe ran past our time budget after certifying 968 of the 162,170 compiled modules. The coordinator still awaits formal verification and an independent audit.

10 Related work

Independent re-checking has a long history in Lean: `lean4checker`, the bundled `leanchecker`, external kernels such as nanoda [17] and Lean4Lean [2], and `lake check` [13, 12], following the Lean FRO's agenda for external checking [14]. Parallel and asynchronous proof checking appears in Isabelle [22], Coq [1] and Lean [8, 7]. Kimina and AXLE provide verification services for AI provers [20, 23]. The closest work is a report on the same library, which built it in the cloud, ran extensive independent checking, and proposed kernel computation as the cause of expensive checks from three source examples [18]. We compare checking with writing, measure the cause module by module, account for the whole library's cost, and provide an original-kernel tool.

11 Conclusion

AI has made writing formal proofs fast, and part of the work it saves reappears in the checker: as tactic search on the first check, and as computation on every check. On OpenAI's library, a model writing 1,000 tokens per second outpaces Lean on almost every module. Checking remains the trustworthy step, it has become an expensive one, and it needs to be planned and paid for. ProofServe, the census data and every script behind these numbers are available at https://github.com/yunwei37/proofserve (release v0.1.0; checker `e000ed62`).

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[3] Cerebras. Kimi K2.6: Fast trillion-parameter inference, 2026. 981 output tokens per second measured by Artificial Analysis on 6 May 2026; next-fastest GPU-based provider 6.7 times slower.

[4] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[5] DeepSeek-AI. Deepseek-prover-v1.5: Lean verification artifact, 2024. Current official artifact inspected 1 November 2026, not a remeasurement of the publication.

[6] Lean community. Mathlib4 reference source at d13f23b7. https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612/Mathlib, 2026. Pinned Mathlib subtree; local Git revision and unchanged source checked 1 November 2026. Lexical census excludes comments and strings.

[7] Lean contributors. Lean 4.17.0: Parallel kernel checking, 2025. Kernel checking concurrent with elaboration.

[8] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[9] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[10] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[11] Lean contributors. Lean 4.34.1 runtime memory-budget checks, 2026. Source and matching SDK disassembly inspected 31 October 2026.

[12] Lean contributors. Lean 4.35.0 release notes, 2026.

[13] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[14] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[15] Linux kernel contributors. Control group v2: memory controller, 2026. Authoritative kernel documentation for memory.max, accounting and OOM semantics; accessed 31 October 2026.

[16] Linux man-pages contributors. getrusage(2): process resource-usage semantics, 2026. RUSAGE SELF aggregates usage across process threads; accessed 31 October 2026.

[17] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[18] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[19] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[20] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[21] SemiAnalysis InferenceX. GB300 NVL72 vs GB200 NVL72 inference performance on DeepSeek-V4-Pro 1.6T, 2026. NVIDIA Dynamo with vLLM, FP4, 8K input and 1K output; 6,182 tokens per second per GPU at 27 tokens per second per user; measured 22 May 2026.

[22] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[23] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.